DATA 211 – Introduction to Data Science
Topic 2c: Pandas and Scikit-learn

Question bank: 40 multiple choice + 10 programming

Questions from DATA 211 / ISE 291 old exams and quizzes (AI_Work). **[new]** marks new questions written in the same style.

Part A: Multiple choice

Q1.

Topic 4 practice questions

Pandas key data structure is called?

(A) nd-array

(B) DataFrame

(C) Both of the above

(D) None of the above

Answer: (B). The DataFrame (2D, labeled) is the main pandas structure; nd-array belongs to NumPy.

My answer

Q2.

Mid exam v important questions

What is a Series in pandas?

(A) A 2-dimensional labeled array

(B) A 2-dimensional labeled list

(C) A 1-dimensional labeled array

(D) A collection of Data frames

Answer: (C). A Series is a one-dimensional labeled array that can hold any data type with an index.

My answer

Q3.

Topic 4 practice questions

The label associated with a particular row of a DataFrame is called its _____.

(A) Data value

(B) Index

(C) Value

(D) None of the above

Answer: (B). Row labels form the index of the DataFrame.

My answer

Q4.

223 Midterm

Which of the following code lines could be used to print the number of rows and columns of a DataFrame?

(A) `print(f'{len(df.index)}, {len(df.columns)}')`

(B) `print(f'{len(df.rows)}, {len(df.columns)}')`

(C) `print(df.shape)`

(D) Both A & C options

Answer: (D). `len(df.index)` and `len(df.columns)` give rows and columns, and `df.shape` gives (rows, columns); `df.rows` does not exist.

My answer

Q5.

Q2B-WithSolution (Quiz 02, T252)

In pandas, what will `df.info()` display?

(A) Summary statistics for numerical columns

(B) Data types, non-null counts, and memory usage

(C) The number of missing values only

(D) Correlation between all numeric columns

Answer: (B). `info()` lists each column with its non-null count and dtype, plus memory usage.

My answer

Q6.

Mid exam v important questions

Consider the following output of `df.info()`. How many data samples, variables, and missing values are there?

```
RangeIndex: 25 entries, 0 to 24
Data columns (total 7 columns):
 0  Country       25 non-null
 1  Population    25 non-null
 2  Overall Life  24 non-null
 3  Male Life     25 non-null
 4  Female Life   25 non-null
 5  Continent     23 non-null
 6  Subregion     25 non-null
```

(A) samples: 25, variables: 7, missing values: 3

(B) samples: 7, variables: 25, missing values: 3

(C) samples: 25, variables: 7, missing values: 0

(D) samples: 175, variables: 7, missing values: 3

Answer: (A). 25 entries (rows), 7 columns; missing = (25−24) + (25−23) = 3.

My answer

Q7.

223 Midterm

Given a dataset in `"secretData.xlsx"`. Which of the following codes will display the last 10 records, first 5 records, and 7 random records, RESPECTIVELY?

(A)
```
df = pd.read_csv('secretData.xlsx')
display(df.tail(10))
display(df.head(5))
display(df.sample(7))
```

(B)
```
df = pd.read_excel('secretData.xlsx')
display(df.head(5))
display(df.tail(10))
display(df.sample(7))
```

(C)
```
df = pd.read_excel('secretData.xlsx')
display(df.tail(10))
display(df.head())
display(df.sample(7))
```

(D)
```
df = pd.read_excel('secretData.xlsx')
display(df.tail(10))
display(df.sample(7))
display(df.head())
```

Answer: (C). `tail(10)`, then `head()` (default 5 rows), then `sample(7)`, in that order, reading the Excel file with `read_excel`.

My answer

Q8.

F08 Quiz02

What is the type of the object returned by `df["column_name"]`?

(A) `pandas.DataFrame`

(B) `pandas.Series`

(C) `numpy.ndarray`

(D) `list`

Answer: (B). Selecting one column with single brackets returns a Series (double brackets `df[['col']]` return a DataFrame).

My answer

Q9. 223 Midterm

In which of the following ways can a column `"C1"` in a DataFrame (`df`) be displayed/extracted?

(A) `df.C1`

(B) `df["C1"]`

(C) `df.loc[:, "C1"]`

(D) All the given options are correct

Answer: (D). Attribute access works because `C1` is a valid Python name; bracket and `.loc` access always work.

My answer

Q10. QuizAZ (Quiz 2, Version C)

Which of the following statements about pandas DataFrames is true?

(A) `df.count()` returns the number of non-null values in each column.

(B) `df.index` returns the number of rows in the DataFrame.

(C) `df.count()` returns the total number of rows, including missing values.

(D) `df.index()` returns the column labels of the DataFrame.

Answer: (A). `count()` ignores NaN; `df.index` is the row-label object (not a number), and column labels are `df.columns`.

My answer

Q11. Mid exam v important questions

In Pandas, how does the `loc` method differ from `iloc` when accessing data in a DataFrame?

(A) `loc` is used for label-based indexing, while `iloc` is used for integer-based indexing.

(B) Both `loc` and `iloc` are used for label-based indexing.

(C) `loc` is used for integer-based indexing, while `iloc` is used for label-based indexing.

(D) Both `loc` and `iloc` are used for integer-based indexing.

Answer: (A). `loc` uses row/column labels; `iloc` uses integer positions.

My answer

Q12.

Topic 4 practice questions

Fill in the blank to get the output as 3.

```
import pandas as pnd
S1=pnd.DataFrame([[1, 2, 3, 4],
                  [5, 6, 7, 8],
                  [9,10,11,12],
                  [13,14,15,16]], index = ['a','b','c','d'])
print(_____)
```

(A) `S1.loc['a',2]`

(B) `S1.iloc['a',2]`

(C) `S1.iloc[2,'a']`

(D) `S1.loc[2,'a']`

Answer: (A). Row label `'a'` and column label 2 give 3; `iloc` does not accept the label `'a'`, and there is no row label 2.

My answer

Q13.

Topic 4 practice questions

How many rows/columns will be printed by the last statement of the given code below?

```
import pandas as pnd
S1=pnd.DataFrame([[1, 2, 3, 4],
                  [5, 6, 7, 8],
                  [9,10,11,12],
                  [13,14,15,16]], index = ['a','b','c','d'])
print(S1.loc[:,1:2])
```

(A) one column

(B) two columns

(C) one row

(D) two rows

Answer: (B). `loc` slicing includes both end labels, so columns 1 and 2 are printed (all four rows).

My answer

Q14.

Topic 4 practice questions

What will the following statement display? `df.iloc[2 : 7, : 3]`

(A) Display 2nd to 7th row and first three columns.

(B) Display 3rd to 7th row and first three columns.

(C) Display 2nd to 7th row and first four columns.

(D) Display 3rd to 7th row and first four columns.

Answer: (B). Positions 2 to 6 (end excluded) are the 3rd to 7th rows; `:3` gives positions 0, 1, 2, i.e. the first three columns.

My answer

Q15.

Mid exam v important questions

When using `loc` for slicing a DataFrame `df`, what does the notation `df.loc[1:4, 2:5]` represent? Assume the rows and columns are integer-labeled (0, 1, ...) without headers.

(A) Elements from the second row to the fourth row and from the third column to the fifth column.

(B) Elements from the second row to the fifth row and from the third column to the sixth column.

(C) Elements from the first row to the fourth row and from the second column to the fifth column.

(D) Elements from the first row to the fifth row and from the second column to the sixth column.

Answer: (B). Labels 1 to 4 (inclusive) are the 2nd to 5th rows; labels 2 to 5 (inclusive) are the 3rd to 6th columns.

My answer

Q16.

Q2B-WithSolution (Quiz 02, T252)

When using the `loc` indexer to slice a DataFrame `df`, what does the expression `df.loc[2:6, 'A':'D']` represent?

(A) Rows 2 to 6 (exclusive) and columns A to D (exclusive)

(B) Rows 2 to 6 (inclusive) and columns B to D (exclusive)

(C) Rows 2 to 6 (inclusive) and columns A to D (inclusive)

(D) Rows 2 to 6 (exclusive) and columns A to D (inclusive)

Answer: (C). With `loc`, both the row and the column slices include their end labels.

My answer

Q17.

Quiz_2 (docx)

Assume a DataFrame `df` of 10 rows and 6 columns with no headers in the data file. Which of the options below can print the top 5 rows of `df`?

(A) `print(df.loc[0:4,:])`

(B) `print(df.loc[0:5])`

(C) `print(df.iloc[0:4,:])`

(D) `print(df.iloc[:,0:5])`

Answer: (A). Labels 0 to 4 inclusive are 5 rows. (B) gives 6 rows, (C) gives 4 rows, and (D) slices columns, not rows.

My answer

Q18.

Quiz2AnswerMS (Quiz 02, T251)

Given a DataFrame `df` with columns `'c1'`, `'c2'`, `'c3'`, `'c4'`, and `'c5'` (default integer row index), you want to extract the first 10 rows from the second to fifth columns using `.loc`. Which code snippet should you use?

(A) `df.loc[0:9, 'c2':'c5']`

(B) `df.iloc[0:9, 'c2':'c5']`

(C) `df.loc[0:9, 'c1':'c4']`

(D) `df.loc[1:10, 'c2':'c6']`

Answer: (A). Row labels 0–9 (inclusive) are the first 10 rows, and `'c2':'c5'` are the 2nd to 5th columns. `iloc` cannot take column names.

My answer

Q19.

223 Midterm

Given a dataset in `"dataset1.xlsx"`, read with `df = pd.read_excel('dataset1.xlsx', header=None)`. Which of the following will slice and display the first ten rows of the first 4 columns using both `.iloc` and `.loc`?

(A) `display(df.iloc[0:9,0:3])` and `display(df.loc[0:9,0:3])`

(B) `display(df.iloc[0:10,0:4])` and `display(df.loc[0:11,0:5])`

(C) `display(df.iloc[0:10,0:4])` and `display(df.loc[0:10,0:4])`

(D) `display(df.iloc[0:10,0:4])` and `display(df.loc[0:9,0:3])`

Answer: (D). `iloc` excludes the end (0–9 and 0–3); `loc` includes it, so labels `0:9` and `0:3` give the same block.

My answer

Q20.

F08 Quiz02

A DataFrame `df` contains columns `['A', 'B', 'C', 'D', 'E']` in that exact order. The code `display(df.loc[:, 'A':'E':2])` will result in displaying columns:

(A) `['A', 'C', 'E']`

(B) `['A', 'C']`

(C) `['B', 'D']`

(D) `['A', 'B', 'C', 'D', 'E']`

Answer: (A). The label slice runs from A to E inclusive with step 2: A, C, E.

My answer

Q21.

Sample Mid

Given a DataFrame with 10 rows labeled from 0 to 9, and 4 columns labeled as C2, C3, C4, and C5. What will be the return of `df.loc[:2,'C2':'C4']` and `df.iloc[:2,0:3]`?

(A) Both will return the same exact rows and columns

(B) `loc` will return more columns and rows than `iloc`

(C) They will return the same exact columns, but `loc` will return more rows

(D) `loc` will return less rows and more columns

Answer: (C). Both give columns C2, C3, C4; `loc` returns rows 0, 1, 2 (3 rows) while `iloc` returns rows 0, 1 (2 rows).

My answer

Q22.

Mid exam v important questions

How can you get the labels of rows where a column value is greater than 10?

(A) `df.iloc[df['column'] > 10].index`

(B) `df.loc[df['column'] > 10].index`

(C) `df.iloc[df['column'] > 10].columns`

(D) `df.loc[df['column'] > 10].columns`

Answer: (B). `loc` accepts the boolean Series and `.index` gives the row labels; `iloc` raises an error with a boolean Series, and `.columns` gives column names.

My answer

Q23.

TOPIC 4 & 5 (Quiz 2, Sec 4)

What is the purpose of the following code?

```
seleted_rows = (df['Province']=='Eastern') & (df['Job Type']=='White-collar')
```

(A) To select all rows where Province is Eastern or Job Type is White-collar

(B) To select all rows where Province is Western and Job Type is Blue-collar

(C) To select all rows where Province is Eastern and Job Type is White-collar

(D) To select all rows where Province is Western or Job Type is Blue-collar

Answer: (C). `&` is element-wise AND of the two conditions.

My answer

Q24.

Mid exam v important questions

How can you display rows in a DataFrame `df` where the `'Status'` field is `'Active'` and the `'Score'` is less than or equal to 75?

(A)
```
selected_rows = (df['Status'] == 'Active') | (df['Score'] <= 75)
display(df.iloc[selected_rows,:])
```

(B)
```
selected_rows = (df['Status'] == 'Active') & (df['Score'] <= 75)
display(df.loc[selected_rows,:])
```

(C)
```
selected_rows = (df['Status'] == 'Active') | (df['Score'] <= 75)
display(df.loc[selected_rows,:])
```

(D)
```
selected_rows = (df['Status'] == 'Active') & (df['Score'] <= 75)
display(df.iloc[:,selected_rows])
```

Answer: (B). "and" needs `&`, and the boolean mask goes in the row position of `loc`.

My answer

Q25.

Exam (11) (Quiz 2, T251 key)

How can you select rows from a pandas DataFrame where the `"age"` column is greater than 30?

(A) `df.loc[df["age"] > 30, :]`

(B) `df.rows("age > 30")`

(C) `df.select_rows("age > 30")`

(D) `df.iloc[df["age"] > 30, :]`

Answer: (A). Boolean selection works with `loc`; `rows`/`select_rows` do not exist and `iloc` does not accept a boolean Series.

My answer

Q26.

Exam (11) (Quiz 2, T251 key)

How can you use a lambda function to apply a transformation to a column named `"price"` in a pandas DataFrame called `df`?

(A) `df.map(lambda x:  x * 0.9, "price")`

(B) `df.transform(lambda x:  x * 0.9, "price")`

(C) `df.apply(lambda x:  x * 0.9, "price")`

(D) `df["price"].apply(lambda x:  x * 0.9)`

Answer: (D). Select the column first, then `apply` the lambda to each of its values.

My answer

Q27.

Quiz_2 (docx)

Which of the following code will sort the DataFrame `df` ascendingly by `"col4"` and then descendingly by `"col1"`?

(A) `df.sort_values(by=['col1', 'col4'], ascending=[False, True])`

(B) `df.sort_values(by=['col4', 'col1'], ascending=[False, True])`

(C) `df.sort_values(by=['col1', 'col4'], ascending=[True, False])`

(D) `df.sort_values(by=['col4', 'col1'], ascending=[True, False])`

Answer: (D). The first key is col4 (ascending = True), the second is col1 (ascending = False); the lists match position by position.

My answer

Q28.

Mid exam v important questions

What does the `inplace=True` parameter do when used with `sort_values()`?

(A) It creates a copy of the sorted DataFrame.

(B) It sorts the DataFrame without saving changes.

(C) It permanently modifies the original DataFrame without creating a copy.

(D) It only prints the sorted DataFrame without sorting the original DataFrame.

Answer: (C). With `inplace=True` the original DataFrame is changed and the method returns `None`.

My answer

Q29.

Quiz_2 (docx)

The code below will:

```
index_row = df[df['education'] == 'unknown'].index
df.drop(index_row, inplace=True)
```

(A) Delete rows with unknown permanently

(B) Drop all the rows in column “education”

(C) Delete rows with unknown not permanently

(D) Drop the column “education”

Answer: (A). The labels of the rows with `'unknown'` are dropped (axis = 0 by default), and `inplace=True` makes it permanent.

My answer

Q30.

Final exam practice 2

Which code segment finds the average `readingScore` per each combination of `gender` and `race`?

(A)
```
df.pivot_table(values=['gender'], index=['readingScore'],
    columns=['race'], aggfunc='mean')
```

(B)
```
df.pivot_table(values=['race'], index=['gender'],
    columns=['readingScore'], aggfunc='sum')
```

(C)
```
df.pivot_table(values=['readingScore'], index=['gender'],
    columns=['race'], aggfunc='mean')
```

(D)
```
df.pivot_table(values=['mean'], index=['gender'],
    columns=['race'], aggfunc='sum')
```

Answer: (C). The averaged quantity goes in `values`, the two grouping fields go in `index` and `columns`, and `aggfunc='mean'`.

My answer

Q31.

TOPIC 4 & 5 (Quiz 02)

For Pandas DataFrames, by default `df.describe()` provides summary for:

(A) Categorical columns.

(B) Numerical columns.

(C) Both A & B.

(D) No default output for `df.describe()` method

Answer: (B). Without arguments, `describe()` summarises only the numeric columns; `include='O'` is needed for object columns.

My answer

Q32.

QuizAZ (Quiz 2, Version C)

When the `df.describe()` method is applied to numeric columns, it does not provide information about which of the following?

(A) Mode

(B) Minimum

(C) Mean

(D) Standard deviation

Answer: (A). Numeric `describe()` gives count, mean, std, min, 25%, 50%, 75%, max, but not the mode.

My answer

Q33.

Final Exam T241 (Code A)

Consider a DataFrame `df` with a mix of categorical and numerical variables.

```
print(df.describe(include='object').loc['unique',:])
```

What will be the output of the above line of code?

(A) The count of values in each column of the DataFrame

(B) The number of categories in each categorical column

(C) The mode of each categorical column

(D) The frequency of the category which occurs the most in each column of the DataFrame

Answer: (B). For object columns `describe` returns count, unique, top, freq; the `'unique'` row is the number of distinct categories.

My answer

Q34.

[new]

What does the following code print?

```
import pandas as pd
s = pd.Series(['1', '2', '3'])
print(s.astype(int).sum())
```

(A) 6

(B) 123

(C) 3

(D) An error, because the values are strings

Answer: (A). `astype(int)` converts the strings to integers, so the sum is $1 + 2 + 3 = 6$ (without it, `sum` would concatenate to `'123'`).

My answer

Q35.

223 Midterm

In Pandas DataFrames, we use _____ methods to find the number of null rows and non-null rows for each column, RESPECTIVELY.

(A) `df.count()` and `df.isna().sum()`

(B) `df.isna().sum()` and `len(df.index)`

(C) `df.isna().sum()` and `df.count()`

(D) `df.isna().count()` and `df.sum()`

Answer: (C). `isna().sum()` counts the NaN values per column; `count()` counts the non-null values per column.

My answer

Q36.

Final exam practice 2

The application of the following line of code is:
`train_test_split(X, y, test_size=0.3, random_state=42)`

(A) To calculate the predicted values of y based on a model.

(B) To separate part of the data for testing while keeping the rest for model fitting.

(C) To perform the MISO OLS closed-form solution.

(D) To transform the data of a numeric variable such that the mean is zero and the standard deviation is one.

Answer: (B). It holds out 30% of the rows as a test set and keeps 70% for training.

My answer

Q37.

Final Exam T241 (Code A)

What is the right code for splitting the data into train and test sets with ratio of 75%:25%, respectively? (In every option `X = df.iloc[:,:-1].values` and `y = df.iloc[:, -1].values`.)

(A)
```
X_train, y_train, X_test, y_test =
train_test_split(X, y, test_size=0.75, random_state=25)
```

(B)
```
X_train, X_test, y_train, y_test =
train_test_split(X, y, test_size=0.75, random_state=25)
```

(C)
```
X_train, y_train, X_test, y_test =
train_test_split(X, y, test_size=0.25, random_state=25)
```

(D)
```
X_train, X_test, y_train, y_test =
train_test_split(X, y, test_size=0.25, random_state=25)
```

Answer: (D). `train_test_split` returns the outputs in the order X_train, X_test, y_train, y_test, and `test_size=0.25` keeps 75% of the data for training.

My answer

Q38. Final exam practice 1

What is the purpose of the `fit()` method in Scikit-learn's classification algorithms?

(A) To preprocess the input data

(B) To evaluate the performance of the model

(C) To train the model on the input data

(D) To make predictions on new data

Answer: (C). `model.fit(X_train, y_train)` learns the model parameters; `predict` makes predictions afterwards.

My answer

Q39. Final Exam T241 (Code A)

Consider the following code for training a classifier (assume `X_train, X_test, y_train, y_test` are defined):

```
from sklearn.tree import DecisionTreeClassifier
model = DecisionTreeClassifier(random_state=0, criterion='entropy', splitter='best')
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

How would you calculate the accuracy of the model on the test set?

(A) `accuracy = accuracy_score(y_train, y_pred)`

(B) `accuracy = accuracy_score(X_train, y_test)`

(C) `accuracy = accuracy_score(y_test, y_pred)`

(D) `accuracy = accuracy_score(X_test, y_test)`

Answer: (C). `accuracy_score` compares the true test labels with the predicted labels.

My answer

Q40. Final 291

Which of the following statements is NOT true about the confusion matrix?

(A) The non-diagonal elements are preferred to be zeros

(B) The diagonal elements are preferred to be very high

(C) The diagonal elements are preferred to be zeros

(D) Both A and B

Answer: (C). The diagonal holds the correct predictions (e.g. `[[10 0 0] [0 9 0] [0 0 11]]` for a perfect model), so it should be high, not zero.

My answer

Part B: Programming

Q41.

Midterm-213 (Problem C)

A medical student is researching the profile of liver patients. A sample of the records is given below.

```
import pandas as pd
data = pd.DataFrame({
    'Age': [65, 62, 61, 37, 45, 27, 37, 58],
    'Gender': ['Female', 'Male', 'Male', 'Male', 'Female', 'Male', 'Female', 'Male'],
    'Total_Protiens': [6.8, 7.5, 8.5, 9.5, 8.6, 8.5, 9.2, 6.8],
    'Aspartate': [18, 100, 231, 54, 44, 54, 30, 20],
    'Diagnosed': ['Yes', 'Yes', 'Yes', 'Yes', 'Yes', 'No', 'No', 'Yes'],
    'Severity': ['High', 'Medium', 'Medium', 'High', 'High', 'Low', 'High', 'Low']})
```

(a) Provide the statistical summary of the numeric fields and of the non-numeric fields (`Gender`, `Diagnosed`, `Severity`).
(b) Display the patient records which have `Total_Protiens` greater than 8.3, whose `Aspartate` value is above 33, and whose `Severity` value is `High` or `Medium`.

Answer:

```
# (a)
print(data.describe())                    # numeric fields
print(data.describe(include='object'))    # non-numeric fields
# (b)
selected_rows = ((data['Total_Protiens'] > 8.3) & (data['Aspartate'] > 33)
                 & ((data['Severity'] == 'High') | (data['Severity'] == 'Medium')))
print(data.loc[selected_rows, :])         # rows 2, 3, 4
```

My answer

Q42.

Set_2 Midterm (Problem C)

Use the following student data.

```
import pandas as pd
df = pd.DataFrame({
    'STU ID': [202010100, 202010357, 202010750, 202011111,
               202011378, 202012001, 202012456, 202013789],
    'Class level': ['sophomore', 'freshman', 'senior', 'senior',
                    'senior', 'junior', 'senior', 'senior'],
    'Department': ['ME', 'EE', 'ISE', 'ISE', 'ISE', 'SWE', 'ISE', 'EE'],
    'Gender': ['M', 'F', 'M', 'M', 'F', 'M', 'M', 'M'],
    'CHEM101': [42, 50, 83, 73, 84, 90, 75, 95]})
```

(a) Display the total number of rows and columns, the name of each column, and the data type assigned by Python to each column.
(b) Display a random sample of 3 rows.
(c) Display all the unique values in the column `Department`.
(d) Display the IDs of all the senior-level male students from the ISE department who scored 75 or more in `CHEM101`.

Answer:

```
# (a)
df.info()
# (b)
print(df.sample(3))
# (c)
print(df['Department'].unique())
# (d)
selected_rows = ((df['Class level'] == 'senior') & (df['Department'] == 'ISE')
                 & (df['Gender'] == 'M') & (df['CHEM101'] >= 75))
print(df.loc[selected_rows, 'STU ID'])    # 202010750, 202012456
```

My answer

Q43.

Set_2 Midterm (D2)

Use the following data.

```
import pandas as pd
df = pd.DataFrame({
    'STU ID': [202010100, 202010357, 202010750, 202011111],
    'Distance': ['122 KM', '725 KM', '556 KM', '273 KM'],
    'MATH101': [86.0, None, None, 51.0]})
```

(a) Column `STU ID` is read as `int64`. Convert its values to a categorical (string/object) type. *Hint: use `str()` to cast an integer into a string.*
(b) Column `Distance` is read as an object (text) type. Convert its values to a numerical (int) type. Print `df.dtypes` before and after the changes.

Answer:

```
print(df.dtypes)
# (a)
df['STU ID'] = df['STU ID'].apply(lambda x: str(x))    # or df['STU ID'].astype(str)
# (b)
df['Distance'] = df['Distance'].apply(lambda x: int(x.replace(' KM', '')))
print(df.dtypes)
print(df)
```

My answer

Q44.

Midterm-213 (D-1)

Use the following data.

```
import pandas as pd
import numpy as np
df = pd.DataFrame({
    'Gender': ['M', 'F', None, 'M', 'F', 'M'],
    'Age': [23, np.nan, 35, 21, np.nan, 27],
    'Albumin_Globulin_Ratio': [0.9, 0.74, np.nan, 1.0, 0.4, 0.89],
    'Diagnosed': ['Yes', 'Yes', 'No', 'Yes', 'No', 'Yes']})
```

(a) Print the number of missing values in each column.
(b) Print the names of the columns that have one or more missing values.
(c) Replace the missing observations using imputation: the mean for numeric columns and the mode for non-numeric columns. Confirm that no missing values remain.

Answer:

```
# (a)
print(df.isnull().sum())
# (b)
null_columns = df.columns[df.isnull().any()]
print(null_columns.tolist())   # ['Gender', 'Age', 'Albumin_Globulin_Ratio']
# (c)
for c in null_columns:
    if pd.api.types.is_numeric_dtype(df[c]):
        df[c] = df[c].fillna(df[c].mean())
    else:
        df[c] = df[c].fillna(df[c].mode()[0])
print(df)
print(df.isnull().sum().sum())   # 0
```

My answer

Q45.

Set_2 Midterm (D5)

Use the following data.

```
import pandas as pd
df = pd.DataFrame({
    'STU ID': [202010100, 202010357, 202010750, 202011111, 202011378],
    'Class level': ['sophomore', 'freshman', 'junior', 'senior', 'freshman']})
```

Create a new column `Class` that gets its values from column `Class level` as follows: `"freshman"` → 1, `"sophomore"` → 2, `"junior"` → 3, and `"senior"` → 4. Display the DataFrame.

Answer:

```
class_mapping = {'freshman': 1, 'sophomore': 2, 'junior': 3, 'senior': 4}
df['Class'] = df['Class level'].map(class_mapping)
print(df)          # Class = [2, 1, 3, 4, 1]
```

My answer

Q46.

[new]

Two fruit price lists are given.

```
import pandas as pd
df1 = pd.DataFrame({'Fruits': ['Mango', 'Banana', 'Grapes', 'Apple', 'Orange'],
                    'Price': [60, 40, 75, 100, 65]})
df2 = pd.DataFrame({'Fruits': ['Apple', 'Orange', 'Papaya', 'Pineapple', 'Mango'],
                    'Price': [120, 60, 30, 70, 50]})
```

(a) Join `df1` and `df2` on `Fruits` using a left, an inner and an outer join, and print each result. How many rows does each join have?
(b) Concatenate `df1` and `df2` row-wise with a new continuous index and print the shape.
(c) Concatenate `df1` column-wise with a new DataFrame `Discount = [5, 7, 10, 8, 6]`.

Answer:

```
# (a)
left  = pd.merge(df1, df2, how='left',  on='Fruits')    # 5 rows
inner = pd.merge(df1, df2, how='inner', on='Fruits')    # 3 rows (Mango, Apple, Orange)
outer = pd.merge(df1, df2, how='outer', on='Fruits')    # 7 rows
print(left, inner, outer, sep='\n\n')
# (b)
rows = pd.concat([df1, df2], axis=0, ignore_index=True)
print(rows.shape)                                        # (10, 2)
# (c)
discount = pd.DataFrame({'Discount': [5, 7, 10, 8, 6]})
print(pd.concat([df1, discount], axis=1))
```

My answer

Q47. [new]

Quiz scores are stored in wide format.

```
import pandas as pd
df = pd.DataFrame({'Student': ['Ali', 'Sara', 'Omar'],
                   'Quiz1': [8, 9, 7],
                   'Quiz2': [6, 10, 9]})
```

(a) Use `pd.melt` to transform `df` to long format with columns `Student`, `Quiz`, `Score`.
(b) Use `pivot` to reshape the long DataFrame back to wide format (one row per student, one column per quiz).
(c) Print the mean of all the scores.

Answer:

```
# (a) wide -> long
long_df = pd.melt(df, id_vars='Student', var_name='Quiz', value_name='Score')
print(long_df)                          # 6 rows
# (b) long -> wide
wide_df = long_df.pivot(index='Student', columns='Quiz', values='Score')
print(wide_df)
# (c)
print(long_df['Score'].mean())          # 8.1667
```

My answer

Q48. [new]

Use the following data.

```
import pandas as pd
df = pd.DataFrame({'Fruits': ['Mango', 'Apple', 'Banana', 'Orange', 'Apple', 'Mango'],
                   'Quantity': [40, 20, 25, 10, 35, 15],
                   'Price': [80, 100, 50, 70, 95, 85]})
```

(a) Print how many times each fruit appears.
(b) Print the two rows with the largest `Price`.
(c) Sort the DataFrame by `Price` in descending order, reset the index (dropping the old one) and store it as `df2`.
(d) In `df2`, rename `Quantity` to `QTY`, then drop the column `QTY` and the rows with index 0 and 1. Print the result.

Answer:

```
# (a)
print(df['Fruits'].value_counts())
# (b)
print(df.nlargest(2, 'Price'))
# (c)
df2 = df.sort_values('Price', ascending=False).reset_index(drop=True)
# (d)
df2 = df2.rename(columns={'Quantity': 'QTY'})
df2 = df2.drop(columns=['QTY'])       # or df2.drop(['QTY'], axis=1)
df2 = df2.drop([0, 1], axis=0)
print(df2)                            # 4 rows, columns Fruits and Price
```

My answer

Q49. [new]

Use the following data.

```
import pandas as pd
df = pd.DataFrame({'Fruits': ['Mango', 'Apple', 'Banana', 'Orange'],
                   'Quantity': [40, 20, 25, 10],
                   'Price': [80, 100, 50, 70]})
```

(a) Change `Price` to `float64`, then print its sum, mean, median, standard deviation and cumulative sum.
(b) Add a new column `Total` = `Quantity` × `Price`.
(c) Using `apply`, print the range (max − min) of the columns `Quantity` and `Price`.
(d) Add a column `Date` with 4 consecutive days starting on 2023-01-01, set it as the index, and print the mean `Quantity` for every 2-day period.

Answer:

```
# (a)
df['Price'] = df['Price'].astype('float64')
print(df['Price'].sum(), df['Price'].mean(), df['Price'].median(), df['Price'].std())
print(df['Price'].cumsum())                    # 80, 180, 230, 300
# (b)
df['Total'] = df['Quantity'] * df['Price']
# (c)
print(df[['Quantity', 'Price']].apply(lambda col: col.max() - col.min()))   # 30, 50
# (d)
df['Date'] = pd.date_range('20230101', periods=4)
df_ts = df.set_index('Date')
print(df_ts['Quantity'].resample('2D').mean())   # 30.0, 17.5
```

My answer

Q50. [new]

Using scikit-learn and the built-in Iris dataset:
(a) Load the dataset into a DataFrame `X` (feature names as column names) and a Series `y` (targets).
(b) Split the data into 80% training and 20% testing with `random_state=42` and print the shapes.
(c) Scale the features with `StandardScaler` (fit on the training set only).
(d) Train a logistic regression model, predict the test set, and print the accuracy, the confusion matrix and the classification report.

Answer:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import pandas as pd
# (a)
iris = load_iris()
X = pd.DataFrame(iris.data, columns=iris.feature_names)
y = pd.Series(iris.target)
# (b)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)  # (120,4) (30,4) (120,)
    (30,)
# (c)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
# (d)
model = LogisticRegression()
model.fit(X_train_scaled, y_train)
y_pred = model.predict(X_test_scaled)
print("Accuracy:", accuracy_score(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

My answer